# Algoritmos de Balanceamento de Árvores

AVL Tree, Red-Black Tree e B-Tree: conceitos, rotações, comparação e aplicações práticas

Pesquisa — CP4 (Dynamic Programming / Estruturas de Dados)

Bruno Gama Masseiras | RM562526

Kauai Rosa de Assis Rocha | RM563256

Lucas Gilmar Pedro da Silva | RM566231

Setembro de 2026

# 1. Introdução

Árvores de busca binária (Binary Search Trees — BST) estão entre as estruturas de dados mais utilizadas para organizar informações que precisam ser buscadas, inseridas e removidas de forma eficiente. O desempenho dessas operações depende diretamente da altura da árvore: quanto mais "baixa e larga" ela for, mais rápidas tendem a ser as operações. O problema é que uma BST comum não garante esse formato — dependendo da ordem em que os elementos são inseridos, ela pode se degenerar em uma estrutura semelhante a uma lista encadeada, perdendo toda a vantagem que a organização em árvore deveria proporcionar.

Este trabalho tem como objetivo estudar o problema do desbalanceamento em árvores de busca e as principais estratégias criadas para evitá-lo. São apresentadas três estruturas de balanceamento — AVL Tree, Red-Black Tree e B-Tree — explicando como cada uma garante que a altura da árvore permaneça controlada, quais mecanismos (em especial as rotações) utilizam para isso, e em que cenários reais essas estruturas são aplicadas. Ao final, é apresentado um quadro comparativo entre as três estruturas e um exemplo prático de uso.

# 2. Árvores de Busca e o Problema do Desbalanceamento

## 2.1 O que é uma árvore de busca binária (BST)

Uma árvore de busca binária é uma estrutura de dados hierárquica em que cada nó possui no máximo dois filhos (esquerdo e direito) e obedece à chamada propriedade da BST: para qualquer nó, todos os valores armazenados na subárvore esquerda são menores do que o valor do nó, e todos os valores da subárvore direita são maiores. Essa propriedade é o que permite que a busca seja feita de forma eficiente, descartando metade das possibilidades a cada comparação, de forma semelhante à busca binária em vetores ordenados.

## 2.2 Busca, inserção e remoção

- **Busca:** inicia-se pela raiz e, a cada nó, compara-se o valor procurado com o valor armazenado. Se for menor, a busca segue para a subárvore esquerda; se for maior, segue para a direita; se for igual, o elemento foi encontrado. O processo termina quando o valor é localizado ou quando se chega a um nó nulo (elemento não existe).
- **Inserção:** segue a mesma lógica da busca até encontrar a posição (um ponteiro nulo) em que o novo valor deveria estar, e o insere ali como uma nova folha.
- **Remoção:** possui três casos — remover um nó folha (basta excluí-lo); remover um nó com apenas um filho (o filho assume o lugar do nó removido); e remover um nó com dois filhos (substitui-se o valor do nó pelo seu sucessor ou predecessor em ordem, e então remove-se esse sucessor/predecessor, que necessariamente terá no máximo um filho).

## 2.3 Árvore balanceada versus desbalanceada

Diz-se que uma árvore está **balanceada** quando, para cada nó, a diferença de altura entre suas subárvores esquerda e direita é pequena (tipicamente limitada por uma constante, como 1). Nesse caso, a altura total da árvore cresce proporcionalmente a log(n), sendo n o número de elementos. Uma árvore está **desbalanceada** quando essa diferença de altura entre subárvores é grande, podendo levar a árvore a se aproximar de uma altura próxima de n no pior caso — ou seja, uma estrutura tão ineficiente para busca quanto uma lista encadeada.

## 2.4 Como a ordem de inserção causa desbalanceamento

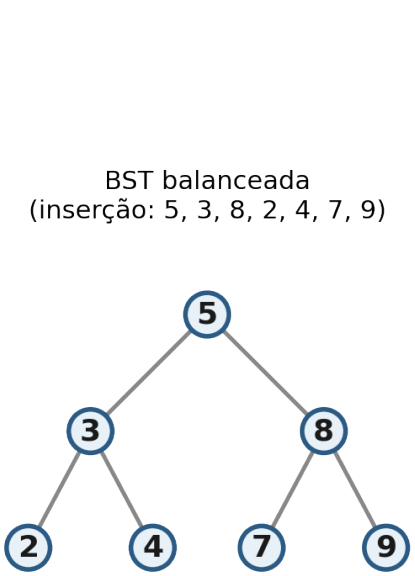
Uma BST convencional não realiza nenhum ajuste estrutural durante as inserções: cada elemento é simplesmente encaixado na posição indicada pela busca. Isso significa que a forma final da árvore depende inteiramente da ordem de chegada dos elementos. Se os valores forem inseridos em uma sequência já ordenada (crescente ou decrescente), cada novo elemento se tornará filho único do anterior, e a árvore degenera em uma lista encadeada, com altura igual ao número de elementos.

## 2.5 Impacto do desbalanceamento na complexidade

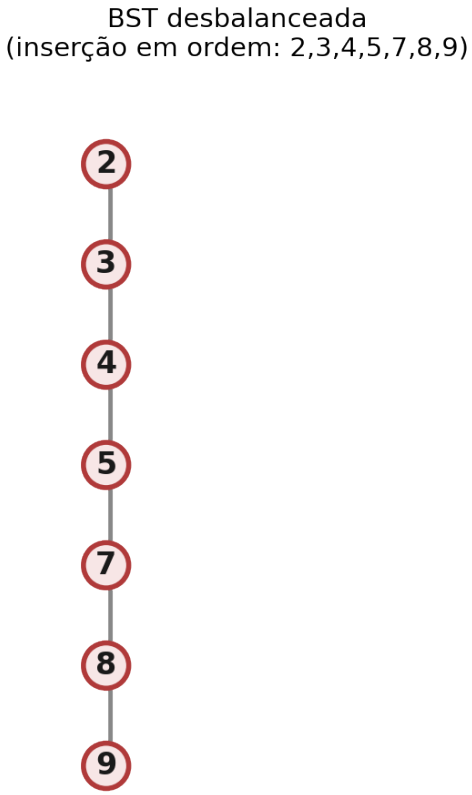
Em uma BST balanceada, busca, inserção e remoção custam O(log n), pois cada comparação elimina metade da árvore restante. Já em uma árvore totalmente desbalanceada, essas mesmas operações passam a custar O(n) no pior caso, pois pode ser necessário percorrer, um a um, todos os elementos até encontrar o alvo — exatamente como em uma busca sequencial em uma lista. É esse risco de degradação de desempenho que motiva o uso de estruturas de balanceamento.

## 2.6 Exemplo: de balanceada a desbalanceada

O exemplo abaixo ilustra o contraste: à esquerda, a inserção dos valores 5, 3, 8, 2, 4, 7 e 9 (em uma ordem que naturalmente distribui os elementos) produz uma árvore de altura baixa. À direita, os mesmos sete valores, inseridos em ordem estritamente crescente (2, 3, 4, 5, 7, 8, 9), resultam em uma árvore completamente desbalanceada, em que cada nó só possui filho à direita — degenerando em uma lista encadeada.




*Árvore balanceada (altura 3)*



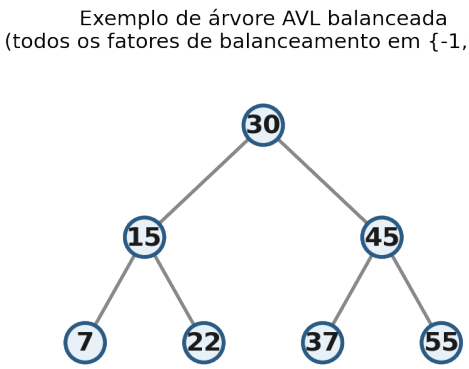

*Mesma quantidade de nós, desbalanceada (altura 7)*

# 3. Balanceamento de Árvores

Balancear uma árvore significa aplicar, durante as próprias operações de inserção e remoção, ajustes estruturais que impeçam a árvore de crescer de forma desproporcional em qualquer um de seus ramos. O objetivo é garantir que a altura da árvore permaneça sempre da ordem de log(n), preservando o desempenho O(log n) nas operações de busca, inserção e remoção mesmo no pior caso — algo que uma BST convencional não garante. As estruturas de balanceamento diferem principalmente na forma como detectam o desequilíbrio e no mecanismo usado para corrigi-lo.

## 3.1 AVL Tree

A AVL Tree, criada por Adelson-Velsky e Landis em 1962, foi a primeira estrutura de dados autobalanceada descrita na literatura. Cada nó mantém um **fator de balanceamento**, definido como a diferença entre a altura da subárvore direita e a da subárvore esquerda. Uma árvore AVL exige que esse fator esteja sempre em {-1, 0, 1} para todos os nós. Após cada inserção ou remoção, o algoritmo sobe pela árvore recalculando os fatores de balanceamento e, ao encontrar um nó cujo fator saiu desse intervalo, aplica uma rotação (simples ou dupla) para restaurar o equilíbrio.

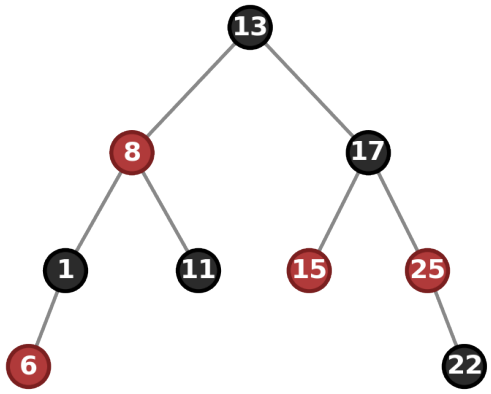



*Árvore AVL — cada nó respeita o fator de balanceamento entre -1 e 1*

## 3.2 Red-Black Tree

A árvore Rubro-Negra é outra BST autobalanceada, amplamente usada em bibliotecas de programação (por exemplo, nas implementações de std::map e std::set do C++, e no TreeMap do Java). Em vez de controlar alturas exatas como a AVL, ela associa uma cor — vermelho ou preto — a cada nó e mantém um conjunto de cinco propriedades, entre elas: a raiz é sempre preta; um nó vermelho nunca tem filho vermelho; e todo caminho da raiz até uma folha nula contém o mesmo número de nós pretos (chamado de altura preta). Essas regras garantem que o caminho mais longo da árvore nunca seja maior que o dobro do caminho mais curto, o que já é suficiente para manter a altura em O(log n), ainda que a árvore não fique tão rigidamente balanceada quanto uma AVL.

Exemplo de árvore Rubro-Negra
(nós pretos e vermelhos; raiz sempre preta)

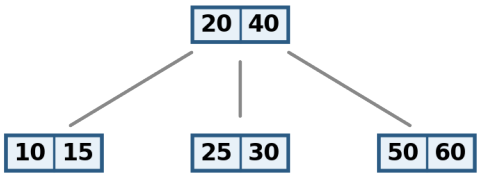



*Árvore Rubro-Negra — nós pretos e vermelhos alternados conforme as regras de coloração*

## 3.3 Terceira estrutura escolhida: B-Tree

Diferentemente da AVL e da Rubro-Negra, a B-Tree não é binária: cada nó pode armazenar várias chaves e ter mais de dois filhos. Ela foi projetada especificamente para cenários em que os dados residem em memória secundária (discos), pois minimiza o número de acessos a disco ao concentrar várias chaves em um mesmo nó/página. Uma B-Tree de ordem m permite que cada nó tenha entre ceil(m/2) e m filhos (exceto a raiz), mantendo automaticamente a árvore balanceada: quando um nó recebe mais chaves do que o permitido, ele é dividido (split) e uma chave é promovida ao nó pai; quando um nó fica com poucas chaves após uma remoção, ele pode pegar emprestada uma chave de um irmão ou se fundir (merge) com ele. Como todas as folhas ficam sempre no mesmo nível, a altura da árvore permanece controlada e proporcional a log_m(n).

Exemplo de B-Tree de ordem 3
(cada nó guarda até 2 chaves e 3 filhos)



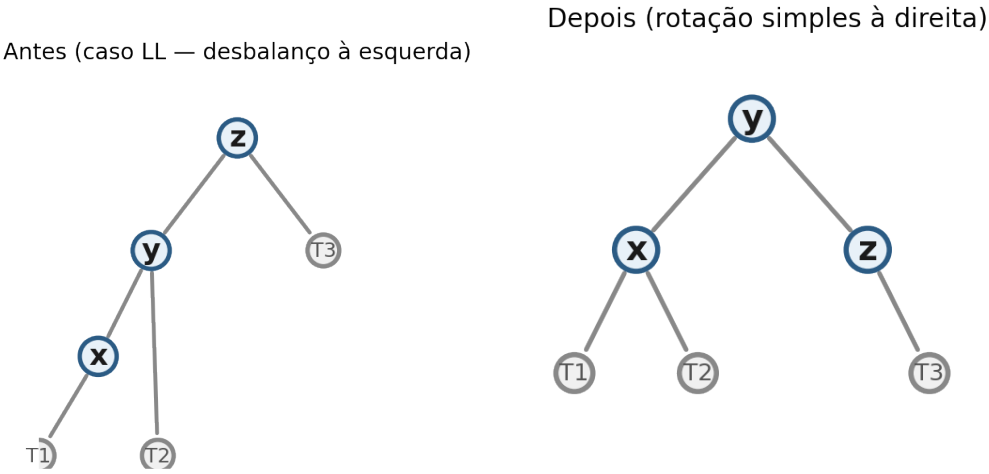

*B-Tree de ordem 3 — cada nó guarda até duas chaves, mantendo todas as folhas no mesmo nível*

# 4. Rotações

Rotações são operações locais de reorganização da árvore que trocam a posição de um nó com um de seus filhos, preservando a propriedade da BST (a ordenação dos valores) mas alterando a altura das subárvores envolvidas. É por meio das rotações que estruturas como a AVL e a Rubro-Negra corrigem um desbalanceamento detectado após uma inserção ou remoção, sem precisar reconstruir a árvore inteira — cada rotação custa apenas O(1), tornando a correção extremamente eficiente.

## 4.1 Rotação simples à direita (caso LL)

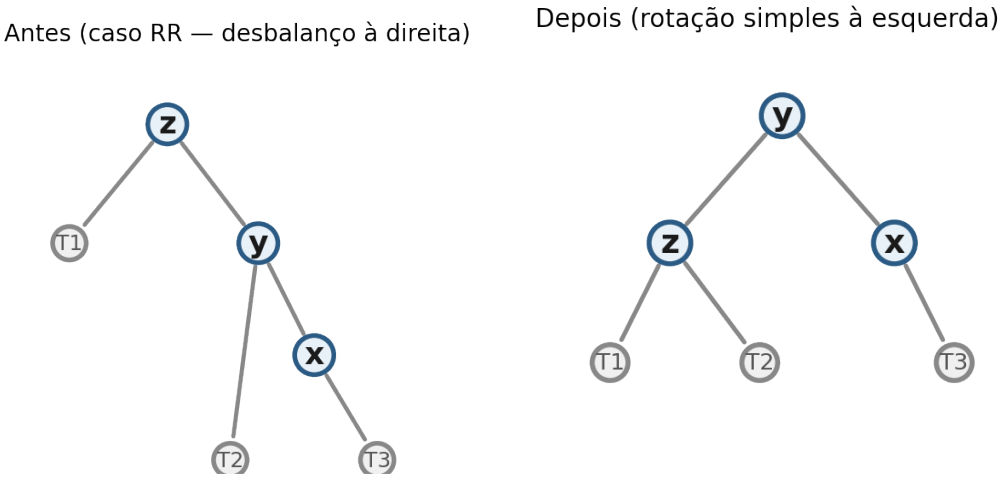
Ocorre quando a subárvore esquerda de um nó z está mais alta do que deveria, especificamente devido a uma inserção na subárvore esquerda do filho esquerdo (y). O nó y sobe para o lugar de z, z se torna filho direito de y, e a subárvore T2 (antigo filho direito de y) passa a ser o filho esquerdo de z.




*Antes da rotação* *Depois da rotação simples à direita*

## 4.2 Rotação simples à esquerda (caso RR)

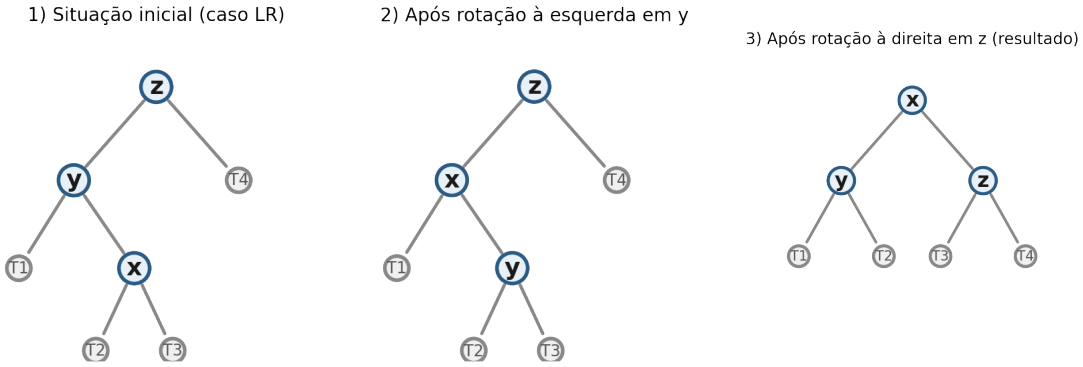
É o espelho da rotação anterior: ocorre quando a subárvore direita está mais alta devido a uma inserção na subárvore direita do filho direito (x). O nó y (filho direito de z) sobe para o topo, z se torna seu filho esquerdo, e a subárvore intermediária T2 passa para z.




*Antes da rotação* *Depois da rotação simples à esquerda*

### 4.3 Rotação dupla esquerda-direita (caso LR)

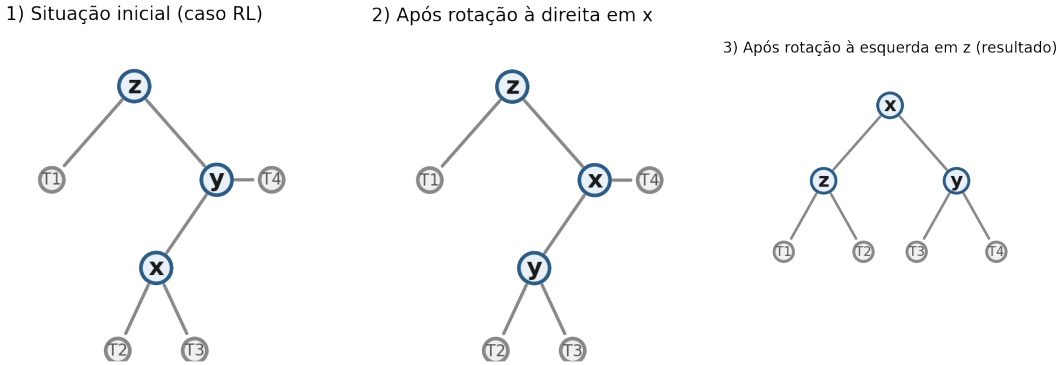
Ocorre quando o desbalanceamento vem da subárvore direita do filho esquerdo — uma única rotação simples não resolveria o problema. Resolve-se aplicando primeiro uma rotação à esquerda em y (transformando o caso em um LL) e, em seguida, uma rotação à direita em z.




*1) Antes* *2) Rotação à esquerda em y* *3) Rotação à direita em z*

### 4.4 Rotação dupla direita-esquerda (caso RL)

É o espelho do caso anterior: o desbalanceamento vem da subárvore esquerda do filho direito. Aplica-se primeiro uma rotação à direita em x (transformando o caso em um RR) e depois uma rotação à esquerda em z.




*1) Antes* *2) Rotação à direita em x* *3) Rotação à esquerda em z*

Vale notar que a árvore Rubro-Negra utiliza exatamente os mesmos quatro tipos de rotação descritos acima; a diferença está no critério que decide quando aplicá-las (baseado em cores e violações das propriedades rubro-negras) e não na mecânica da rotação em si. Já a B-Tree não usa rotações: seu mecanismo de balanceamento é baseado em divisão (split) e fusão (merge) de nós.

## 5. Comparação das Estruturas

O quadro a seguir resume as principais diferenças entre AVL Tree, Red-Black Tree e B-Tree quanto à forma de balanceamento, custo das operações e cenários de aplicação.

| Critério | AVL Tree | Red-Black Tree | B-Tree |
|---|---|---|---|
| Como mantém o balanceamento | Fator de balanceamento (-1,0,1) em cada nó; recalculado após cada operação | Coloração dos nós (vermelho/preto) obedecendo 5 propriedades estruturais | Divisão (split) e fusão (merge) de nós; todas as folhas no mesmo nível |
| Busca | O(log n) — árvore rigidamente balanceada | O(log n) — levemente menos rígida que a AVL | O(log_m n) — poucos níveis, vários acessos por nó |
| Inserção | O(log n); pode exigir 1 rotação (simples ou dupla) | O(log n); no máx. 2 rotações, mas várias recolorações | O(log_m n); pode propagar splits até a raiz |
| Remoção | O(log n); pode exigir rotações em cascata até a raiz | O(log n); rebalanceamento por recoloração e rotações | O(log_m n); pode exigir empréstimos ou merges em cascata |
| Complexidade no pior caso | O(log n) garantido | O(log n) garantido | O(log_m n) garantido |
| Necessidade de rotações | Frequentes, porém baratas (O(1)) | Menos frequentes que na AVL | Não utiliza rotações |
| Vantagens | Busca muito eficiente por ser rigidamente balanceada; ótima quando há muitas buscas e poucas inserções/remoções | Inserção/remoção mais baratas em média (menos reestruturação); muito usada em bibliotecas de propósito geral | Reduz drasticamente o número de acessos a disco; ideal para grandes volumes de dados persistidos |
| Desvantagens | Inserção e remoção podem custar mais rotações do que a Red-Black em média | Balanceamento menos estrito pode deixar buscas levemente mais lentas que na AVL | Maior complexidade de implementação; nós maiores consomem mais memória por nó |
| Aplicações | Bancos de dados e aplicações com forte predominância de buscas (ex.: dicionários em memória) | std::map/std::set (C++), TreeMap/TreeSet (Java), escalonador de processos do Linux (CFS) | Índices de bancos de dados relacionais (MySQL/InnoDB, PostgreSQL), sistemas de arquivos (NTFS, ext4/HFS+ variantes B+Tree) |

# 6. Aplicações Práticas

Árvores balanceadas são usadas sempre que é preciso garantir desempenho previsível em operações de busca, inserção e remoção, independentemente da ordem em que os dados chegam — algo que uma BST comum não assegura. A seguir, alguns exemplos concretos de uso.

## 6.1 Bibliotecas de linguagens de programação

As estruturas ordenadas de coleções como std::map e std::set, do C++, e TreeMap/TreeSet, do Java, são implementadas internamente com árvores Rubro-Negras. Isso garante que operações como inserir, buscar ou remover uma chave em um mapa ordenado custem O(log n) mesmo que o programador insira as chaves em ordem crescente ou decrescente — situação em que uma BST comum se degeneraria completamente.

## 6.2 Bancos de dados e sistemas de arquivos

Sistemas de gerenciamento de banco de dados relacionais, como o MySQL (motor InnoDB) e o PostgreSQL, utilizam variações de B-Tree (em geral B+Tree) para implementar os índices das tabelas. Como o acesso a disco é muito mais lento que o acesso à memória, a B-Tree é preferida nesse contexto por concentrar várias chaves em cada nó/página, reduzindo o número de leituras de disco necessárias para localizar um registro. Sistemas de arquivos como o NTFS também usam B-Trees para organizar diretórios e permitir buscas rápidas por nome de arquivo.

## 6.3 Escalonamento de processos

O escalonador de processos padrão do kernel Linux, chamado CFS (Completely Fair Scheduler), organiza os processos prontos para execução em uma árvore Rubro-Negra ordenada pelo tempo de processador já consumido por cada processo. Isso permite que o escalonador encontre rapidamente o processo que menos recebeu CPU (o próximo a ser executado) e reinsira processos de forma eficiente conforme o tempo passa.

## 6.4 Por que preferir uma árvore balanceada a uma BST comum

Em todos os cenários acima, a entrada de dados não é controlada pela estrutura: um usuário pode inserir registros em ordem alfabética, um índice pode ser criado a partir de uma tabela já ordenada, ou processos podem ser criados e destruídos em qualquer sequência. Uma BST convencional deixaria o desempenho do sistema à mercê dessa ordem de chegada, podendo degradar para O(n) por operação. As estruturas balanceadas eliminam esse risco, garantindo O(log n) (ou O(log_m n), no caso da B-Tree) independentemente da ordem de inserção — um requisito essencial para sistemas de uso geral, que não podem assumir nada sobre como os dados serão inseridos.

# 7. Conclusão

O estudo das árvores de busca binária evidencia um problema simples de enunciar, mas com grande impacto prático: sem nenhum mecanismo de controle, a ordem de inserção dos dados pode transformar uma estrutura projetada para buscas em O(log n) em algo tão lento quanto uma lista encadeada. As estruturas de balanceamento — AVL Tree, Red-Black Tree e B-Tree — resolvem esse problema por meio de mecanismos distintos, porém com o mesmo objetivo: manter a altura da árvore controlada. A AVL prioriza um balanceamento rígido por meio de fatores de altura e rotações, favorecendo cenários com muitas buscas. A Red-Black relaxa esse rigor em troca de

rebalanceamentos mais baratos, sendo a escolha padrão em bibliotecas de propósito geral. Já a B-Tree abandona a estrutura binária em favor de nós com múltiplas chaves, otimizando o acesso a dados em disco. Conhecer as diferenças entre essas estruturas — e, principalmente, saber em que contexto cada uma se sobressai — é fundamental para projetar sistemas eficientes, sejam eles bibliotecas de programação, bancos de dados ou escalonadores de sistemas operacionais.

# Referências Bibliográficas


CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009. Capítulos 12 (Binary Search Trees), 13 (Red-Black Trees) e 18 (B-Trees).

ADELSON-VELSKY, G. M.; LANDIS, E. M. An algorithm for the organization of information. *Soviet Mathematics Doklady*, v. 3, p. 1259–1263, 1962.

BAYER, R.; MCCREIGHT, E. Organization and maintenance of large ordered indices. *Acta Informatica*, v. 1, n. 3, p. 173–189, 1972.

GEEKSFORGEEKS. AVL Tree. Disponível em: https://www.geeksforgeeks.org/avl-tree-set-1-insertion/. Acesso em: set. 2026.

GEEKSFORGEEKS. Red-Black Tree. Disponível em: https://www.geeksforgeeks.org/red-black-tree-set-1-introduction-2/. Acesso em: set. 2026.

GEEKSFORGEEKS. Introduction of B-Tree. Disponível em: https://www.geeksforgeeks.org/introduction-of-b-tree-2/. Acesso em: set. 2026.

MySQL 8.0 Reference Manual — InnoDB and MyISAM Index Statistics Collection. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/index-extensions.html. Acesso em: set. 2026.

TORVALDS, L. et al. *Linux kernel documentation — CFS Scheduler*. Disponível em: https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html. Acesso em: set. 2026.

SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011. Capítulo sobre Balanced Search Trees (Red-Black BSTs e B-Trees).